

```

{
  "nbformat": 4,
  "nbformat_minor": 0,
  "metadata": {
    "colab": {
      "provenance": []
    },
    "kernelspec": {
      "name": "python3",
      "display_name": "Python 3"
    },
    "language_info": {
      "name": "python"
    }
  },
  "cells": [
    {
      "cell_type": "code",
      "source": [
        "import subprocess, sys\n",
        "\n",
        "def _pip_install(*pkgs):\n",
        "    try:\n",
        "        subprocess.run([sys.executable, \"-m\", \"pip\", \"install\", \"-q\",
*pkgs], check=True)\n",
        "    except Exception as e:\n",
        "        print(f\"(pip install skipped/failed for {pkgs}: {e})\")\n",
        "\n",
        "_HAVE_OPENAI = False\n",
        "try:\n",
        "    import openai\n",
        "    _HAVE_OPENAI = True\n",
        "except Exception:\n",
        "    _pip_install(\"openai>=1.0.0\")\n",
        "    try:\n",
        "        import openai\n",
        "        _HAVE_OPENAI = True\n",
        "    except Exception:\n",
        "        _HAVE_OPENAI = False\n",
        "\n",
        "try:\n",
        "    import nest_asyncio\n",
        "    nest_asyncio.apply()\n",
        "except Exception:\n",
        "    try:\n",
        "        _pip_install(\"nest_asyncio\")\n",
        "        import nest_asyncio\n",
        "        nest_asyncio.apply()\n",
        "    except Exception:\n",
        "        pass\n",
        "\n",
        "import os\n",
        "import re\n",
        "import json\n",
        "import time\n",
        "import math\n",
        "import asyncio\n",
        "import inspect\n",
        "import textwrap\n",
        "import contextlib\n",
        "import io\n",
        "from dataclasses import dataclass, field\n",
        "from typing import Any, Callable, Optional, Awaitable, get_type_hints\n",
        "\n",

```

```

"\n",
"def banner(title: str) -> None:\n",
"    line = \"â•\n\" * 78\n",
"    print(f\"\\n{line}\\n {title}\\n{line}\")\n",
"\n",
"\n",
"@dataclass\n",
"class ToolCall:\n",
"    \"\"\"A normalized request from the model to run one tool.\"\"\"\n",
"    id: str\n",
"    name: str\n",
"    arguments: dict\n",
"\n",
"\n",
"@dataclass\n",
"class Usage:\n",
"    prompt_tokens: int = 0\n",
"    completion_tokens: int = 0\n",
"\n",
"    @property\n",
"    def total(self) -> int:\n",
"        return self.prompt_tokens + self.completion_tokens\n",
"\n",
"\n",
"@dataclass\n",
"class LLMResponse:\n",
"    \"\"\"The single shape every provider must return.\"\"\"\n",
"    content: Optional[str]\n",
"    tool_calls: list[ToolCall] = field(default_factory=list)\n",
"    finish_reason: str = \"stop\"\n",
"    usage: Usage = field(default_factory=Usage)\n",
"\n",
"\n",
"class Provider:\n",
"    \"\"\"Base class. A provider turns (messages, tools) into an\n",
LLMResponse.\"\"\"\n",
"    name = \"base\"\n",
"\n",
"    async def complete(self, messages: list[dict], tools: list[dict]) ->\n",
LLMResponse:\n",
"        raise NotImplementedError\n",
"\n",
"\n",
"class OpenAICompatibleProvider(Provider):\n",
"    \"\"\"\n",
"    Works with OpenAI and every OpenAI-compatible gateway (OpenRouter,\n",
DeepSeek,\n",
"    Together, vLLM, LM Studio, Ollama's /v1, ...). This mirrors how nanobot\n",
speaks\n",
"    to most providers under the hood.\n",
"    \"\"\"\n",
"    name = \"openai-compatible\"\n",
"\n",
"    def __init__(self, api_key: str, model: str, base_url: Optional[str] =\n",
None):\n",
"        from openai import AsyncOpenAI\n",
"        self.model = model\n",
"        self.client = AsyncOpenAI(api_key=api_key, base_url=base_url)\n",
"\n",
"    async def complete(self, messages: list[dict], tools: list[dict]) ->\n",
LLMResponse:\n",
"        kwargs: dict[str, Any] = {\"model\": self.model, \"messages\":\n",
messages}\n",
"        if tools:\n",
"            kwargs[\"tools\"] = tools\n",

```

```

"         kwargs[\"tool_choice\"] = \"auto\\n\",
"         resp = await self.client.chat.completions.create(**kwargs)\n",
"         choice = resp.choices[0]\n",
"         msg = choice.message\n",
"         calls: list[ToolCall] = []\n",
"         for tc in (msg.tool_calls or []):\n",
"             try:\n",
"                 args = json.loads(tc.function.arguments or \"{}\")\n",
"             except json.JSONDecodeError:\n",
"                 args = {\"_raw\": tc.function.arguments}\n",
"             calls.append(ToolCall(id=tc.id, name=tc.function.name,
arguments=args))\n",
"         usage = Usage(\n",
"             prompt_tokens=getattr(resp.usage, \"prompt_tokens\", 0) or 0,\n",
"             completion_tokens=getattr(resp.usage, \"completion_tokens\", 0) or
0,\n",
"         )\n",
"         return LLMResponse(\n",
"             content=msg.content,\n",
"             tool_calls=calls,\n",
"             finish_reason=choice.finish_reason or \"stop\",\n",
"             usage=usage,\n",
"         )\n",
"     \n",
"     \n",
" class MockProvider(Provider):\n",
"     \"\"\"\n",
"     A deterministic, rule-based \"LLM\" so this entire tutorial runs with NO API
key\n",
"     and NO network â€” letting you watch the agent loop, tool calls, and memory
work.\n",
"     \n",
"     It imitates the ONE thing that matters for the loop: deciding to emit a tool
call\n",
"     (in the exact normalized shape a real model would) and then, once tool
results\n",
"     come back, producing a final natural-language answer. The agent loop cannot
tell\n",
"     it apart from OpenAI â€” that's the whole point of the provider contract.\n",
"     \"\"\"\n",
"     name = \"mock\"\n",
"     \n",
"     def __init__(self, model: str = \"mock-1\"):\n",
"         self.model = model\n",
"     \n",
"     @staticmethod\n",
"     def _last_user_text(messages: list[dict]) -> str:\n",
"         for m in reversed(messages):\n",
"             if m.get(\"role\") == \"user\":\n",
"                 c = m.get(\"content\")\n",
"                 return c if isinstance(c, str) else json.dumps(c)\n",
"         return \"\"\n",
"     \n",
"     @staticmethod\n",
"     def _already_called(messages: list[dict], tool_name: str) -> bool:\n",
"         for m in messages:\n",
"             if m.get(\"role\") == \"assistant\" and m.get(\"tool_calls\"):\n",
"                 for tc in m[\"tool_calls\"]:\n",
"                     if tc[\"function\"][\"name\"] == tool_name:\n",
"                         return True\n",
"         return False\n",
"     \n",
"     @staticmethod\n",
"     def _extract_math(text: str) -> str:\n",
"         \"\"\"Pull the first math-looking chunk out of a sentence (mock-only

```

```

helper).\"\"\"\\n\",
    \"        t = re.sub(r\"square roots? of (\\d+(?:\\.\\d+)?)\", r\"sqrt(\\1)\",
text)\\n\",
    \"        t = t.replace(\"^\", \"**\")\\n\",
    \"        pattern = (r\"(?:sqrt\\((\\d+(?:\\.\\d+)?\\)|\\d+(?:\\.\\d+)?)\")\\n\",
    \"            r\"(?:\\s*(?:\\|\\+\\|-\\|*\\|/\\|)\\s*(?:sqrt\\((\\d+
(?:\\.\\d+)?\\)|\\d+(?:\\.\\d+)?))*\\)\")\\n\",
    \"        m = re.search(pattern, t)\\n\",
    \"        return m.group(0).strip() if m else t.strip()\\n\",
    \"\\n\",
    \"    @staticmethod\\n\",
    \"    def _scan_memory(messages: list[dict]) -> tuple[Optional[str],
Optional[str]]:\\n\",
    \"        \"\"\"Read back simple facts from prior USER turns â€” proves session
memory is\\n\",
    \"        actually being fed to the model (mock-only convenience).\\\"\"\"\\n\",
    \"        name = love = None\\n\",
    \"        for m in messages:\\n\",
    \"            if m.get(\"role\") == \"user\" and isinstance(m.get(\"content\"),
str):\\n\",
    \"                tx = m[\"content\"].lower()\\n\",
    \"                nm = re.search(r\"my name is (\\w+)\", tx)\\n\",
    \"                if nm:\\n\",
    \"                    name = nm.group(1).title()\\n\",
    \"                lv = re.search(r\"i(?:love|like) (\\w+)\", tx)\\n\",
    \"                if lv:\\n\",
    \"                    love = lv.group(1).title()\\n\",
    \"                return name, love\\n\",
    \"\\n\",
    \"    async def complete(self, messages: list[dict], tools: list[dict]) ->
LLMResponse:\\n\",
    \"        await asyncio.sleep(0)\\n\",
    \"        user = self._last_user_text(messages).lower()\\n\",
    \"        tool_names = {t[\"function\"][\"name\"] for t in tools}\\n\",
    \"        usage = Usage(prompt_tokens=sum(len(str(m)) for m in messages) // 4,
completion_tokens=12)\\n\",
    \"\\n\",
    \"        def call(name, args):\\n\",
    \"            return LLMResponse(\\n\",
    \"                content=None,\\n\",
    \"                tool_calls=
[ToolCall(id=f\"call_{name}_{int(time.time()*1000)%100000}\",\\n\",
    \"                    name=name, arguments=args)],\\n\",
    \"                finish_reason=\"tool_calls\",\\n\",
    \"                usage=usage,\\n\",
    \"            )\\n\",
    \"\\n\",
    \"        has_digit = bool(re.search(r\"\\d\", user))\\n\",
    \"        wants_math = has_digit and (\\n\",
    \"            bool(re.search(r\"[\\+\\|-\\|*\\|/\\|^]\", user)) or \"sqrt\" in user\\n\",
    \"            or \"square root\" in user\\n\",
    \"            or any(w in user for w in [\"calculate\", \"compute\", \"evaluate\",
\\\"what is\\\", \\\"what's\\\"])))\\n\",
    \"        if \"calculator\" in tool_names and wants_math and not
self._already_called(messages, \"calculator\"):\\n\",
    \"            return call(\"calculator\", {\"expression\":
self._extract_math(user)})\\n\",
    \"\\n\",
    \"        if \"get_current_time\" in tool_names and not
self._already_called(messages, \"get_current_time\"):\\n\",
    \"            if any(w in user for w in [\"time\", \"date\", \"today\", \"now\",
\\\"o'clock\\\"]):\\n\",
    \"                tz = \"UTC\"\\n\",
    \"                m = re.search(r\"in ([a-zA-Z\\|/ ]+)\", user)\\n\",
    \"                if m:\\n\",

```

```

        cand = m.group(1).strip().title().replace(" ", "_")\n",
        tz = {"Tokyo": "Asia/Tokyo", "Delhi":
\n"Asia/Kolkata",\n",
        \n"New_York": "America/New_York", "London":
\n"Europe/London"}.get(cand, cand)\n",
        return call("get_current_time", {"timezone": tz})\n",
        \n",
        if "remember_fact" in tool_names and not self._already_called(messages,
\n"remember_fact"):\n",
        m = re.search(r"my favorite (?:(programming )?language is (\w+)\n",
user)\n",
        if m:\n",
        return call("remember_fact", {"key": "favorite_language",
\n"value": m.group(1)})\n",
        \n",
        if "recall_fact" in tool_names and not self._already_called(messages,
\n"recall_fact"):\n",
        if any(w in user for w in ["my favorite", "do you remember",
\n"recall", "what did i tell"]):\n",
        key = "favorite_language" if "language" in user else
\n"note"\n",
        return call("recall_fact", {"key": key})\n",
        \n",
        if "run_python" in tool_names and not self._already_called(messages,
\n"run_python"):\n",
        py_kw = any(w in user for w in ["fibonacci", "prime",
\n"factorial", "simulate"])\n",
        py_action = "python" in user and any(\n",
        w in user for w in ["run", "write", "code", "print",
\n"execute", "snippet"])\n",
        if py_kw or py_action:\n",
        if "fibonacci" in user:\n",
        code = ("def fib(n):\n a,b=0,1\n out=[]\n\n",
        \n for _ in range(n):\n out.append(a); a,b=b,a+b\n
return out\n\n",
        \n"print(fib(12))\n",
        elif "prime" in user:\n",
        code = ("primes=[n for n in range(2,50) \n\n",
        \nif all(n%d for d in
range(2,int(n**0.5)+1))]\n\nprint(primes)\n",
        elif "factorial" in user:\n",
        code = "import math; print(math.factorial(10))\n\n",
        else:\n",
        code = "print(sum(range(1,101)))\n\n",
        return call("run_python", {"code": code})\n",
        \n",
        if "web_search" in tool_names and not self._already_called(messages,
\n"web_search"):\n",
        if any(w in user for w in ["search", "look up", "latest", "news
about", "find information"]):\n",
        return call("web_search", {"query":
self._last_user_text(messages)})\n",
        \n",
        if any(p in user for p in ["my name", "who am i", "what do i love",
\n"what i love"]):\n",
        name, love = self._scan_memory(messages)\n",
        bits = []\n",
        if name:\n",
        bits.append(f"your name is {name}")\n",
        if love:\n",
        bits.append(f"you love {love}")\n",
        if bits:\n",
        return LLMResponse(content="From our conversation, " + " and
\n.join(bits) + ".\n",\n",
        \n
        tool_calls=[], finish_reason="stop",

```

```

usage=usage)\n",
    "\n",
    "    tool_outputs = [m[\"content\"] for m in messages if m.get(\"role\") ==
\"tool\"]\n",
    "    if tool_outputs:\n",
    "        joined = \" \".join(tool_outputs)\n",
    "        answer = f\"Based on the tool results, here's what I found:
{joined}\"\n",
    "    elif any(w in user for w in [\"hello\", \"hi\", \"hey\"]):\n",
    "        answer = \"Hello! I'm a mock nanobot agent. Ask me to calculate, tell
time, run Python, or remember things.\"\n",
    "    else:\n",
    "        answer = (\"[mock LLM] I would normally reason about this with a real
model. \"\n",
    "                  \"Set NANOBOT_API_KEY to use a live LLM. For now, try prompts
with math, \"\n",
    "                  \"time, Python, or memory so you can see the tool loop
fire.\\")\n",
    "    return LLMResponse(content=answer, tool_calls=[], finish_reason=\"stop\",
usage=usage)"
    ],
    "metadata": {
        "id": "tfoii-P4bxrW"
    },
    "execution_count": null,
    "outputs": []
},
{
    "cell_type": "code",
    "source": [
        "_PYTYPE_TO_JSON = {str: \"string\", int: \"integer\", float: \"number\", bool:
\"boolean\",
        "list: \"array\", dict: \"object\"}\n",
        "\n",
        "\n",
        "@dataclass\n",
        "class Tool:\n",
        "    name: str\n",
        "    description: str\n",
        "    parameters: dict\n",
        "    func: Callable\n",
        "    is_async: bool\n",
        "\n",
        "    def spec(self) -> dict:\n",
        "        \"\"\"OpenAI-style tool spec the model sees.\"\"\"\n",
        "        return {\"type\": \"function\",
        "                \"function\": {\"name\": self.name,
        "                               \"description\": self.description,
        "                               \"parameters\": self.parameters}}\n",
        "\n",
        "    async def __call__(self, **kwargs) -> str:\n",
        "        try:\n",
        "            result = self.func(**kwargs)\n",
        "            if inspect.isawaitable(result):\n",
        "                result = await result\n",
        "            return result if isinstance(result, str) else json.dumps(result,
default=str)\n",
        "        except Exception as e:\n",
        "            return f\"ERROR running tool '{self.name}': {type(e).__name__}:
{e}\"\n",
        "\n",
        "\n",
        "def tool(func: Optional[Callable] = None, *, name: Optional[str] = None):\n",
        "    \"\"\"Decorator that turns a plain function into a Tool, deriving the JSON schema

```

```

from\n",
"    type hints and the first line of the docstring. Param descriptions can be
added\n",
"    with a simple 'param: description' block in the docstring.\n",
"\n",
"    Example:\n",
"    @tool\n",
"    def calculator(expression: str) -> str:\n",
"        '''Evaluate a math expression and return the result.\n",
"        expression: a math expression like \"2 + 2 * 3\" or \"sqrt(16)\"'''\n",
"        ... \n",
"    \n",
"    def make(f: Callable) -> Tool:\n",
"        hints = get_type_hints(f)\n",
"        sig = inspect.signature(f)\n",
"        doc = inspect.getdoc(f) or \"\"\n",
"        summary = doc.split(\"\\n\", 1)[0].strip() or f.__name__\n",
"\n",
"        param_docs: dict[str, str] = {}\n",
"        for line in doc.splitlines()[1:]:\n",
"            m = re.match(r\"\\s*(\\w+)\\s*:\\s*(.+)\", line)\n",
"            if m and m.group(1) in sig.parameters:\n",
"                param_docs[m.group(1)] = m.group(2).strip()\n",
"\n",
"        props, required = {}, []\n",
"        for pname, p in sig.parameters.items():\n",
"            if pname == \"self\":\n",
"                continue\n",
"            jtype = _PYTYPE_TO_JSON.get(hints.get(pname, str), \"string\")\n",
"            schema = {\"type\": jtype}\n",
"            if pname in param_docs:\n",
"                schema[\"description\"] = param_docs[pname]\n",
"            props[pname] = schema\n",
"            if p.default is inspect.Parameter.empty:\n",
"                required.append(pname)\n",
"\n",
"        parameters = {\"type\": \"object\", \"properties\": props, \"required\":
required}\n",
"        return Tool(name=name or f.__name__, description=summary,\n",
"                    parameters=parameters, func=f,\n",
"                    is_async=inspect.iscoroutinefunction(f))\n",
"\n",
"    return make(func) if func else make\n",
"\n",
"\n",
"class ToolRegistry:\n",
"    def __init__(self):\n",
"        self._tools: dict[str, Tool] = {}\n",
"\n",
"    def add(self, t: Tool) -> None:\n",
"        self._tools[t.name] = t\n",
"\n",
"    def add_function(self, f: Callable) -> None:\n",
"        self.add(tool(f))\n",
"\n",
"    def get(self, name: str) -> Optional[Tool]:\n",
"        return self._tools.get(name)\n",
"\n",
"    def specs(self) -> list[dict]:\n",
"        return [t.spec() for t in self._tools.values()]\n",
"\n",
"    def names(self) -> list[str]:\n",
"        return list(self._tools)\n",
"\n",
"\n",

```

```

"@tool\n",
"def calculator(expression: str) -> str:\n",
"    \"\"\"Evaluate an arithmetic expression and return the numeric result.\n",
"    expression: a math expression, e.g. '2 + 2 * 3', 'sqrt(16)', '2 **
10'\n",
"    allowed = {k: getattr(math, k) for k in dir(math) if not
k.startswith(\"_\")} \n",
"    allowed.update({\"abs\": abs, \"round\": round, \"min\": min, \"max\": max,
\"sqrt\": math.sqrt}) \n",
"    expr = expression.replace(\"^\", \"*\") \n",
"    value = eval(expr, {\"__builtins__\": {}}, allowed) \n",
"    return f\"{expression} = {value}\" \n",
"\n",
"\n",
"@tool\n",
"def get_current_time(timezone: str = \"UTC\") -> str:\n",
"    \"\"\"Return the current date and time for an IANA timezone name.\n",
"    timezone: IANA tz like 'UTC', 'Asia/Tokyo', 'Asia/Kolkata',
'America/New_York'\"\"\" \n",
"    from datetime import datetime \n",
"    try: \n",
"        from zoneinfo import ZoneInfo \n",
"        now = datetime.now(ZoneInfo(timezone)) \n",
"    except Exception: \n",
"        from datetime import timezone as _tz \n",
"        now = datetime.now(_tz.utc) \n",
"        timezone = \"UTC (fallback)\" \n",
"    return f\"Current time in {timezone}: {now.strftime('%Y-%m-%d %H:%M:%S
%Z')}\" \n",
"\n",
"\n",
"@tool\n",
"def run_python(code: str) -> str:\n",
"    \"\"\"Execute a short Python snippet in a restricted namespace and return its
stdout.\n",
"    code: Python source code to run; use print(...) to produce output\"\"\" \n",
"    safe_builtins = {\"print\": print, \"range\": range, \"len\": len, \"sum\":
sum, \"min\": min, \n",
"                    \"max\": max, \"abs\": abs, \"sorted\": sorted, \"enumerate\":
enumerate, \n",
"                    \"list\": list, \"dict\": dict, \"set\": set, \"str\": str,
\"int\": int, \n",
"                    \"float\": float, \"bool\": bool, \"map\": map, \"filter\":
filter, \n",
"                    \"zip\": zip, \"all\": all, \"any\": any, \"round\":
round} \n",
"    import math as _m \n",
"    g = {\"__builtins__\": safe_builtins, \"math\": _m} \n",
"    buf = io.StringIO() \n",
"    try: \n",
"        with contextlib.redirect_stdout(buf): \n",
"            exec(code, g, {}) \n",
"        out = buf.getvalue().strip() \n",
"        return f\"stdout: \n{out}\" if out else \"(ran successfully, no
stdout)\" \n",
"    except Exception as e: \n",
"        return f\"Python error: {type(e).__name__}: {e}\" \n",
"\n",
"\n",
"@tool\n",
"def web_search(query: str) -> str:\n",
"    \"\"\"Search the web for a query and return short result snippets (STUB).\n",
"    query: the search query string\"\"\" \n",
"    return (f\"[stub results for '{query}'] (1) Overview article. (2) Official
docs. \n",

```

```

"          f\"(3) Recent discussion. Swap web_search's body for a real API in
production.\")\n",
"\n",
"\n",
"def estimate_tokens(messages: list[dict]) -> int:\n",
"    \"\"\"Rough token estimate (~4 chars/token) â€” good enough for budgeting
demos.\"\"\"\n",
"    chars = 0\n",
"    for m in messages:\n",
"        chars += len(str(m.get(\"content\") or \"\"))\n",
"        for tc in (m.get(\"tool_calls\") or []):\n",
"            chars += len(json.dumps(tc))\n",
"    return max(1, chars // 4)\n",
"\n",
"\n",
"class Memory:\n",
"    def __init__(self, token_budget: int = 3000):\n",
"        self.token_budget = token_budget\n",
"        self._sessions: dict[str, list[dict]] = {}\n",
"\n",
"    def history(self, session_key: str) -> list[dict]:\n",
"        return self._sessions.setdefault(session_key, [])\n",
"\n",
"    def append(self, session_key: str, message: dict) -> None:\n",
"        self.history(session_key).append(message)\n",
"\n",
"    def extend(self, session_key: str, messages: list[dict]) -> None:\n",
"        self.history(session_key).extend(messages)\n",
"\n",
"    def compact(self, session_key: str) -> int:\n",
"        \"\"\"Drop oldest messages until under the token budget. Returns
#dropped.\"\"\"\n",
"        Keeps tool-call/tool-result pairs consistent by trimming from the front
in\n",
"        whole turns. (nanobot also summarizes; we keep it to trimming for
clarity.)\"\"\"\n",
"        hist = self.history(session_key)\n",
"        dropped = 0\n",
"        while estimate_tokens(hist) > self.token_budget and len(hist) > 2:\n",
"            hist.pop(0)\n",
"            dropped += 1\n",
"        while hist and hist[0].get(\"role\") == \"tool\":\n",
"            hist.pop(0); dropped += 1\n",
"        return dropped"
],
"metadata": {
    "id": "W6AkHJ_abxg9"
},
"execution_count": null,
"outputs": []
},
{
    "cell_type": "code",
    "source": [
        "@dataclass\n",
        "class AgentHookContext:\n",
        "    iteration: int = 0\n",
        "    messages: list[dict] = field(default_factory=list)\n",
        "    response: Optional[LLMResponse] = None\n",
        "    usage: Usage = field(default_factory=Usage)\n",
        "    tool_calls: list[ToolCall] = field(default_factory=list)\n",
        "    tool_results: list[str] = field(default_factory=list)\n",
        "    final_content: Optional[str] = None\n",
        "    stop_reason: Optional[str] = None\n",
        "    error: Optional[Exception] = None

```

```

"\n",
"\n",
"class AgentHook:\n",
"    \"\"\"Subclass and override what you need. All async methods are best-effort
and\n",
"    isolated (one failing hook won't crash the agent).\"\"\"\n",
"    def wants_streaming(self) -> bool:\n",
"        return False\n",
"\n",
"    async def before_iteration(self, context: AgentHookContext) -> None: ... \n",
"    async def on_stream(self, context: AgentHookContext, delta: str) -> None:
... \n",
"    async def on_stream_end(self, context: AgentHookContext, *, resuming: bool) ->
None: ... \n",
"    async def before_execute_tools(self, context: AgentHookContext) -> None:
... \n",
"    async def after_iteration(self, context: AgentHookContext) -> None: ... \n",
"\n",
"    def finalize_content(self, context: AgentHookContext, content: str) -> str:\n",
"        return content\n",
"\n",
"\n",
"    async def _fan_out(hooks: list[AgentHook], method: str, *args, **kwargs) ->
None:\n",
"        for h in hooks:\n",
"            try:\n",
"                await getattr(h, method)(*args, **kwargs)\n",
"            except Exception as e:\n",
"                print(f\"    (hook {type(h).__name__}.{method} error: {e})\")\n",
"\n",
"\n",
"@dataclass\n",
"class Skill:\n",
"    name: str\n",
"    description: str\n",
"    instructions: str = \"\"\n",
"    tools: list[Tool] = field(default_factory=list)\n",
"\n",
"\n",
"class MCPServer:\n",
"    \"\"\"Minimal stand-in for an MCP server exposing named tools.\"\"\"\n",
"    def __init__(self, name: str):\n",
"        self.name = name\n",
"        self._impls: dict[str, dict] = {}\n",
"\n",
"    def register(self, name: str, description: str, parameters: dict, handler:
Callable):\n",
"        self._impls[name] = {\"description\": description, \"parameters\":
parameters, \"handler\": handler}\n",
"\n",
"    def list_tools(self) -> list[dict]:\n",
"        return [{\"name\": n, \"description\": v[\"description\"], \"parameters\":
v[\"parameters\"]}\n",
"                for n, v in self._impls.items()]\n",
"\n",
"    async def call_tool(self, name: str, arguments: dict) -> str:\n",
"        impl = self._impls[name]\n",
"        res = impl[\"handler\"](**arguments)\n",
"        if inspect.isawaitable(res):\n",
"            res = await res\n",
"        return res if isinstance(res, str) else json.dumps(res, default=str)\n",
"\n",
"\n",
"def mcp_tools(server: MCPServer) -> list[Tool]:\n",
"    \"\"\"Adapt every tool on an MCP server into our native Tool objects.\"\"\"

```

```

"    out: list[Tool] = []\n",
"    for spec in server.list_tools():\n",
"        nm = spec[\"name\"]\n",
"\n",
"        async def _runner(_nm=nm, **kwargs):\n",
"            return await server.call_tool(_nm, kwargs)\n",
"\n",
"        out.append(Tool(name=f\"{server.name}__{nm}\",\n",
"            description=f\"[MCP:{server.name}]\n",
{spec['description']}\",\n",
"            parameters=spec[\"parameters\"], func=_runner,\n",
is_async=True))\n",
"    return out\n",
"\n",
"\n",
"@dataclass\n",
"class RunResult:\n",
"    content: str\n",
"    tools_used: list[str] = field(default_factory=list)\n",
"    iterations: int = 0\n",
"    usage: Usage = field(default_factory=Usage)\n",
"    messages: list[dict] = field(default_factory=list)\n",
"\n",
"\n",
"class Agent:\n",
"    def __init__(self, provider: Provider, registry: ToolRegistry, memory:\n",
Memory,\n",
"        system_prompt: str, max_iterations: int = 6, verbose: bool =\n",
True):\n",
"        self.provider = provider\n",
"        self.registry = registry\n",
"        self.memory = memory\n",
"        self.system_prompt = system_prompt\n",
"        self.max_iterations = max_iterations\n",
"        self.verbose = verbose\n",
"\n",
"    def _log(self, *a):\n",
"        if self.verbose:\n",
"            print(*a)\n",
"\n",
"    async def run(self, user_message: str, *, session_key: str = \"default\",\n",
"        hooks: Optional[list[AgentHook]] = None,\n",
"        extra_instructions: str = \"\") -> RunResult:\n",
"        hooks = hooks or []\n",
"\n",
"        system = self.system_prompt\n",
"        if extra_instructions:\n",
"            system += \"\\n\\n\" + extra_instructions\n",
"        self.memory.append(session_key, {\"role\": \"user\", \"content\":\n",
user_message})\n",
"        dropped = self.memory.compact(session_key)\n",
"        if dropped:\n",
"            self._log(f\"  Â· memory compaction dropped {dropped} old\n",
message(s))\n",
"        messages = [{\"role\": \"system\", \"content\": system},\n",
*self.memory.history(session_key)]\n",
"\n",
"        ctx = AgentHookContext(messages=messages)\n",
"        tools_used: list[str] = []\n",
"        total = Usage()\n",
"        final_text = \"\"\n",
"\n",
"        for i in range(1, self.max_iterations + 1):\n",
"            ctx.iteration = i\n",
"            ctx.messages = messages

```

```

        "        await _fan_out(hooks, \"before_iteration\", ctx)\n",
        "\n",
        "        response = await self.provider.complete(messages,
self.registry.specs())\n",
        "        ctx.response = response\n",
        "        total.prompt_tokens += response.usage.prompt_tokens\n",
        "        total.completion_tokens += response.usage.completion_tokens\n",
        "        ctx.usage = total\n",
        "\n",
        "        if response.tool_calls:\n",
        "            ctx.tool_calls = response.tool_calls\n",
        "            self._log(f\" [iter {i}] model requested
{len(response.tool_calls)} tool call(s)\")\n",
        "            messages.append({\n",
        "                \"role\": \"assistant\",\n",
        "                \"content\": response.content,\n",
        "                \"tool_calls\": [{\"id\": tc.id, \"type\": \"function\", \"function\": {\"name\": tc.name, \"arguments\":
json.dumps(tc.arguments)}}]\n",
        "                for tc in response.tool_calls],\n",
        "            })\n",
        "            await _fan_out(hooks, \"before_execute_tools\", ctx)\n",
        "\n",
        "            results: list[str] = []\n",
        "            for tc in response.tool_calls:\n",
        "                t = self.registry.get(tc.name)\n",
        "                if t is None:\n",
        "                    result = f\"ERROR: unknown tool '{tc.name}'\"\n",
        "                else:\n",
        "                    result = await t(**tc.arguments)\n",
        "                tools_used.append(tc.name)\n",
        "                results.append(result)\n",
        "                self._log(f\"      â†³ {tc.name}({tc.arguments}) ->
{result[:120]}\")\n",
        "                messages.append({\"role\": \"tool\", \"tool_call_id\":
tc.id,\n",
        "                    \"content\": result})\n",
        "                ctx.tool_results = results\n",
        "                await _fan_out(hooks, \"after_iteration\", ctx)\n",
        "                continue\n",
        "\n",
        "            final_text = response.content or \"\"\n",
        "            for h in hooks:\n",
        "                try:\n",
        "                    final_text = h.finalize_content(ctx, final_text)\n",
        "                except Exception as e:\n",
        "                    print(f\" (hook {type(h).__name__}.finalize_content error:
{e})\")\n",
        "            ctx.final_content = final_text\n",
        "            ctx.stop_reason = response.finish_reason\n",
        "            await _fan_out(hooks, \"after_iteration\", ctx)\n",
        "            self.memory.append(session_key, {\"role\": \"assistant\", \"content\":
final_text})\n",
        "            break\n",
        "        else:\n",
        "            final_text = \"(stopped: hit max_iterations without a final
answer)\"\n",
        "\n",
        "        return RunResult(content=final_text, tools_used=tools_used,\n",
        "            iterations=ctx.iteration, usage=total,\n",
        "            messages=list(messages))\"
    ],
    \"metadata\": {
        \"id\": \"25Iovy0Ubxab\"
    }

```

```

    },
    "execution_count": null,
    "outputs": []
  },
  {
    "cell_type": "code",
    "source": [
      "DEFAULT_SYSTEM_PROMPT = (\n",
      "    \n\"You are nanobot, a concise, helpful personal AI agent. You can call tools
when \n\n",
      "    \n\"they help. Prefer using a tool over guessing for math, the current time,
running \n\n",
      "    \n\"code, web lookups, or recalling stored facts. After tools run, answer the
user \n\n",
      "    \n\"directly and clearly.\n\n",
      ")\n",
      "\n",
      "\n",
      "\n",
      "class Nanobot:\n",
      "    def __init__(self, provider: Provider, *, system_prompt: str =
DEFAULT_SYSTEM_PROMPT,\n",
      "                token_budget: int = 3000, max_iterations: int = 6, verbose: bool =
True):\n",
      "        self.registry = ToolRegistry()\n",
      "        self.memory = Memory(token_budget=token_budget)\n",
      "        self.skills: dict[str, Skill] = {}\n",
      "        self._loaded_skills: set[str] = set()\n",
      "        self._base_system = system_prompt\n",
      "        self.agent = Agent(provider, self.registry, self.memory,\n",
      "                           system_prompt, max_iterations=max_iterations,
verbose=verbose)\n",
      "        for t in (calculator, get_current_time, run_python, web_search):\n",
      "            self.registry.add(t)\n",
      "\n",
      "        @classmethod\n",
      "        def auto(cls, **kw) -> \"Nanobot\":\n",
      "            \n\n\"Pick a real provider if an API key is set, else the Mock
provider.\n\n",
      "            api_key = os.environ.get(\"NANOBOT_API_KEY\") or
os.environ.get(\"OPENAI_API_KEY\")\n",
      "            model = os.environ.get(\"NANOBOT_MODEL\", \"gpt-4o-mini\")\n",
      "            base_url = os.environ.get(\"NANOBOT_BASE_URL\")\n",
      "            if api_key and _HAVE_OPENAI:\n",
      "                print(f\"â†’ Using live provider: OpenAI-compatible (model={model},
base_url={base_url or 'api.openai.com'})\")\n",
      "                provider: Provider = OpenAICompatibleProvider(api_key, model,
base_url)\n",
      "            else:\n",
      "                why = \"no API key found\" if not api_key else \"openai SDK
unavailable\n",
      "                print(f\"â†’ Using Mock provider ({why}). Set NANOBOT_API_KEY for a
live model.\n",
      "                provider = MockProvider()\n",
      "                return cls(provider, **kw)\n",
      "\n",
      "        def add_tool(self, f: Callable) -> \"Nanobot\":\n",
      "            self.registry.add(tool(f) if not isinstance(f, Tool) else f)\n",
      "            return self\n",
      "\n",
      "        def register_skill(self, skill: Skill) -> \"Nanobot\":\n",
      "            self.skills[skill.name] = skill\n",
      "            return self\n",
      "\n",
      "        def load_skill(self, name: str) -> \"Nanobot\":\n",
      "            \n\n\"Activate a skill: append its instructions and register its

```

```

tools.\\"\"\"\\n",
    "        sk = self.skills[name]\\n",
    "        if name not in self._loaded_skills:\\n",
    "            self.agent.system_prompt += f\"\\n\\n## Skill:
{sk.name}\\n{sk.instructions}\"\\n",
    "            for t in sk.tools:\\n",
    "                self.registry.add(t)\\n",
    "                self._loaded_skills.add(name)\\n",
    "                print(f\"  Â· loaded skill '{name}' ({len(sk.tools)} tool(s))\\n",
    "            return self\\n",
    "\\n",
    "    def connect_mcp(self, server: MCPServer) -> \"Nanobot\\n\":\\n",
    "        for t in mcp_tools(server):\\n",
    "            self.registry.add(t)\\n",
    "            print(f\"  Â· connected MCP server '{server.name}' (+
{len(server.list_tools())} tool(s))\\n",
    "        return self\\n",
    "\\n",
    "    async def run(self, message: str, *, session_key: str = \"sdk:default\\n\",\\n",
    "        hooks: Optional[list[AgentHook]] = None) -> RunResult:\\n",
    "        return await self.agent.run(message, session_key=session_key,
hooks=hooks)\\n",
    "\\n",
    "\\n",
    "class AuditHook(AgentHook):\\n",
    "    \"\"\"Print every tool the model decides to call.\"\"\"\\n",
    "    def __init__(self):\\n",
    "        self.calls: list[str] = []\\n",
    "\\n",
    "    async def before_execute_tools(self, context: AgentHookContext) -> None:\\n",
    "        for tc in context.tool_calls:\\n",
    "            self.calls.append(tc.name)\\n",
    "            print(f\"      [audit] {tc.name}({tc.arguments})\\n",
    "\\n",
    "\\n",
    "class TimingHook(AgentHook):\\n",
    "    \"\"\"Measure how long each LLM iteration takes.\"\"\"\\n",
    "    def __init__(self):\\n",
    "        self._t = 0.0\\n",
    "\\n",
    "    async def before_iteration(self, context: AgentHookContext) -> None:\\n",
    "        self._t = time.perf_counter()\\n",
    "\\n",
    "    async def after_iteration(self, context: AgentHookContext) -> None:\\n",
    "        ms = (time.perf_counter() - self._t) * 1000\\n",
    "        print(f\"      [timing] iteration {context.iteration} took {ms:.1f}
ms\\n\",
    "\\n",
    "\\n",
    "class CensorHook(AgentHook):\\n",
    "    \"\"\"finalize_content runs as a pipeline â€” transform the final
text.\"\"\"\\n",
    "    def finalize_content(self, context: AgentHookContext, content: str) -> str:\\n",
    "        return content.replace(\"secret\\n\", \"***\\n\") if content else content\\n",
    "\\n",
    "\\n",
    "async def demo_basic(bot: Nanobot):\\n",
    "    banner(\"DEMO 1 â€” Basic chat (no tools needed)\\n\",
    "    r = await bot.run(\"Hello! Who are you?\\n\", session_key=\"demo-basic\\n\",
    "    print(\"AGENT:\\n\", r.content)\\n",
    "    print(f\"(iterations={r.iterations}, tools={r.tools_used}, ~tokens=
{r.usage.total})\\n\",
    "\\n",
    "\\n",
    "async def demo_tool_calling(bot: Nanobot):\\n",

```

```

"    banner(\\"DEMO 2 â€" Tool calling: math, time, and Python\\")\\n",
"    for q in [\\"What is 2 ** 10 + sqrt(144)?\\",\\n",
"              \\"What time is it in Tokyo?\\",\\n",
"              \\"Write Python to list the first 12 Fibonacci numbers.\\"]:\\n",
"        print(f\\"\\nUSER: {q}\\")\\n",
"        r = await bot.run(q, session_key=\\\"demo-tools\\")\\n",
"        print(\\"AGENT:\\", r.content)\\n",
"\\n",
"\\n",
"async def demo_multistep(bot: Nanobot):\\n",
"    banner(\\"DEMO 3 â€" Multi-step loop with an audit hook\\")\\n",
"    audit = AuditHook()\\n",
"    q = \\"Calculate 15 * 23, and also tell me the current time in
Asia/Kolkata.\\\"\\n",
"    print(f\\"USER: {q}\\")\\n",
"    r = await bot.run(q, session_key=\\\"demo-multistep\\", hooks=[audit])\\n",
"    print(\\"AGENT:\\", r.content)\\n",
"    print(\\"Tools observed by hook:\\", audit.calls)\\n",
"\\n",
"\\n",
"async def demo_memory(bot: Nanobot):\\n",
"    banner(\\"DEMO 4 â€" Session memory (independent histories per
session_key)\\")\\n",
"    await bot.run(\\"My name is Ada and I love Python.\\", session_key=\\\"user-
ada\\")\\n",
"    await bot.run(\\"My name is Alan and I love Haskell.\\", session_key=\\\"user-
alan\\")\\n",
"    r1 = await bot.run(\\"What's my name and what do I love?\\", session_key=\\\"user-
ada\\")\\n",
"    r2 = await bot.run(\\"What's my name and what do I love?\\", session_key=\\\"user-
alan\\")\\n",
"    print(\\"ADA session â†'\\", r1.content)\\n",
"    print(\\"ALAN session â†'\\", r2.content)\\n",
"    print(\\"(Each session_key kept its own conversation history â€" like
nanobot.)\\")\\n",
"\\n",
"\\n",
"async def demo_skills(bot: Nanobot):\\n",
"    banner(\\"DEMO 5 â€" Skills: load a 'research' capability on demand\\")\\n",
"    research = Skill(\\n",
"        name=\\\"research\\",\\n",
"        description=\\\"Web research workflow\\",\\n",
"        instructions=(\\"When researching, first search the web, then synthesize the
\\\"\\n",
"                    \\"snippets into a short, sourced summary.\\"),\\n",
"        tools=[web_search],\\n",
"    )\\n",
"    bot.register_skill(research).load_skill(\\\"research\\")\\n",
"    r = await bot.run(\\"Search for the latest on retrieval-augmented generation and
summarize.\\",\\n",
"                    session_key=\\\"demo-skills\\")\\n",
"    print(\\"AGENT:\\", r.content)\\n",
"\\n",
"\\n",
"async def demo_mcp(bot: Nanobot):\\n",
"    banner(\\"DEMO 6 â€" MCP-style external tool server\\")\\n",
"    server = MCPServer(\\\"weather\\")\\n",
"    server.register(\\n",
"        name=\\\"forecast\\",\\n",
"        description=\\\"Get a (stub) weather forecast for a city.\\",\\n",
"        parameters={\\"type\\": \\"object\\",\\n",
"                    \\"properties\\": {\\"city\\": {\\"type\\": \\"string\\"}},\\n",
"                    \\"required\\": [\\"city\\"],\\n",
"        handler=lambda city: f\\"Forecast for {city}: 27Â°C, partly cloudy (stub MCP
data).\\",\\n",

```

```

"    )\n",
"    bot.connect_mcp(server)\n",
"    print(\"Registered tools now include:\", [n for n in bot.registry.names() if
\"weather\" in n])\n",
"    t = bot.registry.get(\"weather__forecast\")\n",
"    print(\"Direct MCP tool call â†’\", await t(city=\"Delhi\"))\n",
"\n",
"\n",
"async def demo_streaming_and_finalize(bot: Nanobot):\n",
"    banner(\"DEMO 7 â€” finalize_content pipeline + timing hook\")\n",
"    q = \"Compute sqrt(2) to show the math tool, then reply.\"\n",
"    print(f\"USER: {q}\")\n",
"    r = await bot.run(q, session_key=\"demo-hooks\", hooks=[TimingHook(),
CensorHook()])\n",
"    print(\"AGENT:\", r.content)\n",
"\n",
"\n",
"async def demo_capstone(bot: Nanobot):\n",
"    banner(\"DEMO 8 â€” Capstone: a personal agent juggling tools + memory\")\n",
"    print(\"A short multi-turn 'personal assistant' conversation:\")\n",
"    turns = [\n",
"        \"Remember that my favorite programming language is Python.\",\n",
"        \"What's 144 / 12, and what's my favorite language?\",\n",
"        \"Run Python to print all primes under 50.\"\n",
"    ]\n",
"    for q in turns:\n",
"        print(f\"USER: {q}\")\n",
"        r = await bot.run(q, session_key=\"capstone\", hooks=[AuditHook()])\n",
"        print(\"AGENT:\", r.content, \"\\n\\n\")\n",
"],
"metadata": {\n",
"    "id": "FW6R7LBEbxXm"\n",
"},
"execution_count": null,\n",
"outputs": []\n",
},
{\n",
"    "cell_type": "code",\n",
"    "execution_count": 1,\n",
"    "metadata": {\n",
"        "colab": {\n",
"            "base_uri": "https://localhost:8080/"\n",
"        },\n",
"        "id": "CQMM2xI9Zd9D",\n",
"        "outputId": "d8a2d4f8-b378-4f1b-83ed-263c29e3e736"\n",
"    },\n",
"    "outputs": [\n",
"        {\n",
"            "output_type": "stream",\n",
"            "name": "stdout",\n",
"            "text": [\n",
"                \"\n",
"                \" ðŸŽˆ nanobot-from-scratch â€” building & running the core architecture\",\n",
"                \"\n",
"                \" â†’ Using Mock provider (no API key found). Set NANOBOT_API_KEY for a live\n",
"                model.\",\n",
"                \"Registered tools: ['calculator', 'get_current_time', 'run_python',\n",
"                'web_search', 'remember_fact', 'recall_fact']\"\n",

```


[illegible]

[illegible]

```

" # configure a provider + model in ~/.nanobot/config.json, then:\n",
" from nanobot import Nanobot as RealNanobot\n",
" bot = RealNanobot.from_config()\n",
" result = await bot.run(\"What time is it in Tokyo?\")\n",
" print(result.content)\n",
"Docs: https://github.com/HKUDS/nanobot  â€¢ Python SDK: docs/python-sdk.md\n",
"\n"
]
}
],
"source": [
  "_FACTS: dict[str, str] = {}\n",
  "\n",
  "\n",
  "@tool\n",
  "def remember_fact(key: str, value: str) -> str:\n",
  "    \"\"\"Store a fact in long-term key-value memory.\n",
  "    key: short identifier\n",
  "    value: the value to store\"\"\"\n",
  "    _FACTS[key] = value\n",
  "    return f\"Stored {key} = {value}\"\n",
  "\n",
  "\n",
  "@tool\n",
  "def recall_fact(key: str) -> str:\n",
  "    \"\"\"Recall a previously stored fact by key.\n",
  "    key: the identifier used when storing\"\"\"\n",
  "    return _FACTS.get(key, f\"(no fact stored under '{key}')\")\n",
  "\n",
  "\n",
  "async def main():\n",
  "    banner(\"ðŸŒˆ nanobot-from-scratch  â€” building & running the core\narchitecture\")\n",
  "    bot = Nanobot.auto(verbose=True)\n",
  "    bot.add_tool(remember_fact).add_tool(recall_fact)\n",
  "    print(\"Registered tools:\", bot.registry.names())\n",
  "\n",
  "    await demo_basic(bot)\n",
  "    await demo_tool_calling(bot)\n",
  "    await demo_multistep(bot)\n",
  "    await demo_memory(bot)\n",
  "    await demo_skills(bot)\n",
  "    await demo_mcp(bot)\n",
  "    await demo_streaming_and_finalize(bot)\n",
  "    await demo_capstone(bot)\n",
  "\n",
  "    banner(\"DONE\")\n",
  "    print(textwrap.dedent(\"\"\"\n        You just built nanobot's core: a provider-agnostic agent loop with\n        token-budgeted session memory, lifecycle hooks, skills, and an MCP-style\n        server  â€” the same architecture HKUDS/nanobot ships, kept deliberately\n        small.\n    \"\"\"))\n",
  "\n",
  "    â€” Run the REAL nanobot\n",
  "    !pip install nanobot-ai\n",
  "    # configure a provider + model in ~/.nanobot/config.json, then:\n",
  "    from nanobot import Nanobot as RealNanobot\n",
  "    bot = RealNanobot.from_config()\n",
  "    result = await bot.run(\"What time is it in Tokyo?\")\n",
  "    print(result.content)\n",
  "    Docs: https://github.com/HKUDS/nanobot  â€¢ Python SDK: docs/python-

```

```
sdk.md\n",
    "\n\n\n\n\n",
    "\n",
    "\n",
    "def _go():\n",
    "    try:\n",
    "        asyncio.run(main())\n",
    "    except RuntimeError:\n",
    "        loop = asyncio.get_event_loop()\n",
    "        loop.run_until_complete(main())\n",
    "\n",
    "\n",
    "if __name__ == \"__main__\":\n",
    "    _go()"
]
}
]
```